

Absolutely. Let's continue with **Set 5 — more difficult, real-world GenAI production scenarios**. These are designed so you can practice answering in **3–5 sentences**, which is a good interview style.

■ GenAI 4+ Years Interview — Set 5

201. Your chatbot gives the correct answer but takes 30 seconds. What do you do?

Answer:

"I would measure latency at every stage: query embedding, vector search, reranking, prompt preparation, network calls, and LLM generation. I would identify the actual bottleneck first. Then I could reduce unnecessary context, optimize retrieval, use caching, asynchronous processing, streaming, or a faster model."

202. Your chatbot responds in 1 second for simple questions but 20 seconds for complex questions. Why?

Answer:

"Complex questions may require more retrieval, more chunks, more tokens, or multiple LLM calls. I would trace the request and compare retrieval count, prompt size, and generation time between simple and complex queries."

203. How would you reduce LLM latency without changing the model?

Answer:

"I would reduce input context, reduce unnecessary Top-K results, optimize prompts, limit output tokens, cache repeated requests, use streaming, and avoid unnecessary sequential API calls."

204. You have 50,000 documents but only 500 are relevant to a department. How do you avoid searching everything?

Answer:

"I would use metadata filtering before or during vector search. For example, I could filter by department, tenant, document type, or access level before ranking the results."

205. Why is metadata important in RAG?

Answer:

"Metadata allows us to filter and control retrieval based on business information such as department, tenant, document version, date, access level, and document type. It improves both relevance and security."

■ Document Versioning

206. User asks for the policy that was active in 2024, but the current policy is from 2026. How do you handle it?

Answer:

"I would use the document's effective date and version metadata. Instead of always retrieving the latest document, the query would filter or rank documents based on the requested historical date."

207. How do you maintain historical documents?

Answer:

"I would keep each version with its own document ID or version number and effective date. Old documents can remain available for audit and historical queries while the current version is marked active."

208. Two documents have the same version number. What do you do?

Answer:

"I would enforce unique document/version identifiers during ingestion. If a conflict occurs, the ingestion pipeline should reject or quarantine the document for manual review."

■ Data Quality

209. The PDF was uploaded successfully, but the extracted text is empty. What do you check?

Answer:

"I would check whether the PDF is scanned or image-based, whether the parser supports the document format, and whether OCR is required. I would validate extracted text before continuing to chunking and embedding."

210. The extracted text contains many strange characters. What do you do?

Answer:

"I would inspect the PDF encoding and extraction library. I would clean invalid characters and validate the extracted text before creating embeddings."

211. The document contains headers and footers repeated on every page. What problem can this cause?

Answer:

"Repeated headers and footers can create duplicate information in many chunks and reduce retrieval quality. I would remove or normalize them during preprocessing."

212. The same company logo appears on every page. Does it affect normal text RAG?

Answer:

"If it is only an image and not meaningful text, it generally should not be embedded as text. However, if the visual information is important, a multimodal document-processing approach may be required."

■ Retrieval Failure

213. User asks a question using terminology not found anywhere in the documents. What do you do?

Answer:

"Semantic embeddings may still retrieve relevant information if the concepts are similar. If retrieval confidence remains low, I would use query expansion or rewriting. If no relevant information is found, the system should say 'I don't know.'"

214. What if the question is completely outside the knowledge base?

Answer:

"The system should detect that no relevant context exists using retrieval thresholds or an intent classifier. It should avoid hallucinating and return a controlled response."

215. Should the LLM answer if retrieval returns no documents?

Answer:

"For a knowledge-grounded RAG chatbot, I would normally prevent unsupported generation. The system should say that the information isn't available rather than allowing the model to guess."

216. Retrieval returns relevant chunks, but none contain the complete answer.

Answer:

"I would check whether the chunking strategy is splitting related information. I could increase overlap, retrieve neighboring chunks, or use parent-child retrieval."

■ Reranking

217. Why use a reranker if vector search already works?

Answer:

"Vector search efficiently retrieves candidate documents, but the ranking may not always be optimal. A reranker can perform a more detailed relevance comparison and reorder the candidates before they are sent to the LLM."

218. Does reranking replace vector search?

Answer:

"Usually no. Vector search is often used for efficient candidate retrieval, and reranking is applied to those candidates to improve relevance."

219. Reranking improves accuracy but increases latency. What do you do?

Answer:

"I would measure whether the quality improvement justifies the latency. I could reduce the candidate count, use a faster reranker, cache results, or apply reranking only to complex queries."

■ Hybrid Search

220. Your vector search doesn't find exact product codes. What do you do?

Answer:

"I would introduce hybrid search combining keyword/BM25 search with vector similarity search. Exact identifiers are often better handled by keyword retrieval."

221. How would you combine keyword and vector results?

Answer:

"I would retrieve candidates from both methods and combine or rank them using a suitable scoring strategy. The final candidates can then be reranked before being passed to the LLM."

■ Prompt & Context

222. Your prompt contains 10 pages of instructions. Is that good?

Answer:

"Not necessarily. Excessive instructions can increase token usage and make behavior harder to control. I would simplify the prompt, remove redundant instructions, and clearly define the task, context, constraints, and output format."

223. The model sometimes follows retrieved document instructions instead of your system instructions. Why?

Answer:

"Retrieved documents should be treated as untrusted data rather than instructions. I would clearly separate system instructions from retrieved context and explicitly tell the model not to follow instructions contained inside the documents."

224. How would you structure a RAG prompt?

Answer:

```
System Instructions
  ↓
User Question
  ↓
Retrieved Context
  ↓
Rules:
- Answer only from context
- Don't invent information
- Say "I don't know" if unavailable
- Provide sources
  ↓
Expected Output Format
```

■ Token & Context Window

225. What happens if retrieved context exceeds the model's context window?

Answer:

"The request may fail or important information may be truncated depending on the system. I would reduce unnecessary chunks, rerank them, summarize where appropriate, and control the context size."

226. How do you decide how many chunks to send?

Answer:

"I would evaluate retrieval quality and token usage. I would start with an appropriate Top-K and tune it based on the evaluation dataset rather than choosing an arbitrary number."

227. Why shouldn't you send 50 chunks to the LLM just because they are relevant?

Answer:

"More context isn't always better. It increases token cost, latency, and the possibility of conflicting or distracting information. I would select the smallest amount of context that reliably answers the question."

■ Cost Optimization

228. Your monthly LLM bill doubled. What do you investigate?

Answer:

"I would check request volume, input tokens, output tokens, model usage, retries, duplicate requests, and changes in prompt or retrieval configuration. I would compare the current metrics against the previous period."

229. Your application retries failed LLM requests 5 times. What problem can this cause?

Answer:

"It can increase latency and cost and potentially make rate-limit problems worse. I would use limited retries with exponential backoff and stop retrying errors that are not transient."

230. Why use exponential backoff?

Answer:

"It prevents repeated requests from hitting a failing service immediately. The wait time increases between retries, reducing pressure on the downstream API."

■ Observability

231. What is the difference between logging and monitoring?

Answer:

"Logging records detailed events about individual requests or errors. Monitoring tracks metrics such as latency, error rate, throughput, token usage, and resource utilization over time."

232. What alerts would you configure for a RAG application?

Answer:

I would monitor:

- High API error rate
- High latency
- 429 rate-limit errors
- Vector DB failures
- High token usage
- High cost
- Queue backlog
- High CPU/memory
- Low retrieval quality
- LLM availability

233. How would you detect a sudden quality degradation?

Answer:

"I would monitor evaluation metrics and user feedback and compare them with a baseline. If quality drops after a deployment, I would compare model, prompt, embedding, retrieval, and configuration versions."

■ Security & Authorization

234. Why shouldn't authorization happen only after the LLM generates the response?

Answer:

"Because the unauthorized information has already reached the model. Access control must happen before retrieval so unauthorized content never enters the generation context."

235. Can two users receive different answers for the same question?

Answer:

"Yes. If they have different permissions or tenant data, retrieval should be filtered based on their authorization. The same question can legitimately produce different answers from different accessible knowledge bases."

236. How would you isolate customer data?

Answer:

"I would use tenant IDs and enforce tenant-level filtering in retrieval. I would also apply authorization at the application and data layers and avoid sharing cached responses across tenants."

■ API & Reliability

237. LLM API returns a 500 error. Should you always retry?

Answer:

"Not blindly. I would determine whether the error is transient. For retryable server or network errors, I would use limited retries with exponential backoff. For permanent errors, retrying won't help."

238. LLM API returns a 400 error. Should you retry?

Answer:

"Usually no. A 400 generally indicates an invalid request, so I would inspect the request payload, token limit, parameters, or schema instead of repeatedly sending the same request."

239. What is timeout handling?

Answer:

"A timeout prevents the application from waiting indefinitely for a downstream service. After a timeout, I can retry if appropriate, use a fallback, or return a graceful response."

■ Testing

240. How do you test a RAG application before production?

Answer:

"I would test document ingestion, chunking, retrieval, authorization, prompt behavior, hallucination resistance, API failures, latency, concurrency, and cost. I would use an evaluation dataset with expected answers and relevant sources."

241. What negative test cases would you create?

Answer:

Examples:

- Question not in documents
- Unauthorized document request
- Prompt injection
- Empty query
- Very long query
- Invalid document
- Duplicate document
- Deleted document
- Conflicting documents
- LLM unavailable

242. Why test questions where the answer doesn't exist?

Answer:

"Because a good RAG system should know when it doesn't have sufficient information. These tests help measure hallucination and whether the system correctly returns 'I don't know.'"

■ CI/CD

243. How would you include GenAI testing in CI/CD?

Answer:

"I would maintain a regression evaluation dataset and run automated tests against it when prompts, models, embeddings, or retrieval logic change. A deployment would fail or require review if key quality metrics fall below defined thresholds."

244. Developer changes the system prompt. What should happen before production?

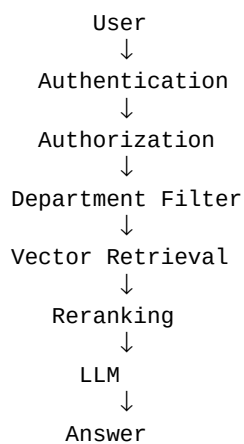
Answer:

"The new prompt should be evaluated against the regression dataset and compared with the previous version for correctness, groundedness, latency, and cost."

■ Architecture Design

245. How would you design RAG for multiple departments?

Answer:



"Each chunk would contain department and access metadata. Retrieval would be filtered according to the authenticated user's permissions."

246. How would you design RAG for multiple companies?

Answer:

"I would use tenant isolation. Each document and chunk would have a tenant ID, and every retrieval request would be filtered by the authenticated tenant. Cache keys, logs, and storage should also respect tenant boundaries."

247. How would you design RAG for 1 million documents?

Answer:

"I would use asynchronous distributed ingestion, batch embedding, scalable vector indexing, metadata filtering, efficient retrieval, reranking, caching, and monitoring. I would separate the ingestion workload from the real-time

query service and auto-scale workers based on workload."

248. How would you handle 1000 PDF uploads at the same time?

Answer:

"I would place upload events into a queue and process them asynchronously with multiple workers. Workers would perform extraction, chunking, embedding, and indexing. I would use retries and a dead-letter queue for failures."

249. How would you prevent ingestion from affecting user queries?

Answer:

"I would separate ingestion workers from the query-serving infrastructure. They can scale independently so heavy document processing doesn't consume resources needed for real-time queries."

■ 250. The Ultimate Interview Question

Your RAG system is failing in production. How do you troubleshoot it?

A strong answer:

***I would first identify the scope and impact of the issue using monitoring and logs. Then I would trace the request through the RAG pipeline: authentication and authorization, query processing, embedding generation, vector retrieval, reranking, prompt construction, LLM API, and response validation.**

>

If retrieval is wrong, I would check chunking, embeddings, metadata filters, Top-K, thresholds, and reranking. If retrieval is correct but the answer is wrong, I would check the prompt, model configuration, context size, and groundedness. If the application is slow, I would check latency for each component.

>

For infrastructure problems, I would check API availability, vector database health, CPU, memory, network, queues, and scaling. After identifying the root cause, I would fix it, validate using regression tests, monitor the deployment, and document the root cause and preventive action."

■ Memorize this troubleshooting chain

User → Auth → Query → Embedding → Retrieval → Reranking → Context → Prompt → LLM → Validation → Response